This node has only one child, so in a *BST* it would just be replaced by its child.
(B) However, in a k-d tree this might cause violating the k-d tree invariants, since moving a node one level up changes the coordinate on which the split is performed, when using it as a pivot.

And indeed, figure 9.18 shows an example where this becomes relevant. The problem is, when we break ties on `insert` and `search` by going right, we implicitly assume an invariant[9]: that for any internal node `N`, no node in its left branch will have the same value for the coordinate used to partition `N`'s subtree. In figure 9.18, this means that no node in the left branch of node `B` has `y` coordinate equal to `N`'s.

If we replace `N` the max of its left branch, however, it is possible that in `N`'s old left branch there was another node with the same `y` coordinate: in our example, that would be the case since there are two nodes with the same, maximal value for `y`, node `I` and node `H`.

By moving node `I` to replace `B`, we would therefore break search, because node `H` would never ever be found by the `search` method in listing 9.3.